

- Explanation:
 - class MyApp extends StatelessWidget → MyApp does not have a state.
 - build() method → Returns the UI structure of the widget.
 - MaterialApp → The root widget of the app.
- c. What is super.key and Why Is It Used?
 - super.key passes the key value to the parent class (StatelessWidget or StatefulWidget).
 - It helps Flutter efficiently identify and update widgets in the widget tree.
 - It's optional, but recommended for better widget optimization and reusability.
- d. Comparison with Java
 - Both Dart and Java use extends to create a subclass.
 - In Java, UI is often defined separately, while in Flutter, UI is created directly within the widget class.

```
public class MainActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

- Dart is more concise and optimized for Flutter's widget-based design.

Step 5 : Overriding Methods with @override

- a. What is @override in Flutter?
 - @override is used when a method in a child class replaces a method from its parent class.
 - In Flutter, @override is often used in build(), which is inherited from StatelessWidget or StatefulWidget.
 - It ensures that the method correctly overrides the parent method.
 - This is an example overriding build() methods :

```
@override
Widget build(BuildContext context) {
```

- Explanation:
 - @override → Tells Flutter that this build() method replaces the parent's build() method.
 - build(BuildContext context) → Defines how the UI should be displayed.
 - Without @override, Dart will still work, but using it improves code clarity and helps avoid mistakes.
- b. Why is @override Important?
 - Ensures that the method is correctly overridden → If the method name or signature is incorrect, Dart will show an error.
 - Makes the code easier to understand → It explicitly indicates that this method is replacing a parent method.

- Prevents accidental mistakes → Without `@override`, if the method is misspelled, Dart might not detect the error.
- c. Comparison: `@override` in Java vs. Flutter
- Both Java and Flutter (Dart) use `@override` to indicate method overriding.
 - This helps avoid mistakes when method signatures don't match. Java requires explicit method signatures, while Dart is more flexible.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- Explanation:
 - `@override` → Ensures `build()` in Child correctly overrides the parent's method.
 - If `build()` was misspelled, Java would throw an error.
- Comparison table :

Feature	Java	Flutter
Usage of <code>@override</code>	Required for overriding methods in OOP	Commonly used in Flutter widgets
Error Prevention	Prevents incorrect method signatures	Helps detect typos in overridden methods
Flexibility	Strict method signatures	More dynamic, but <code>@override</code> improves clarity

Step 6 : Exploring the Build Method and BuildContext

- a. What is the `build()` Method?
- The `build()` method describes how a widget should appear on the screen.
 - It returns a “widget tree”, which is a collection of nested widgets forming the UI.
 - The `build()` method is called whenever the UI needs to be updated.

```
class MyApp extends StatelessWidget {
    @Override
    Widget build(BuildContext context) {
        return MaterialApp(
            home: Scaffold(
                body: Center(
                    child: Text('Hello, Flutter!'),
                ),
            ),
        );
    }
}
```